

ВСТУП

Сучасні інформаційні технології передбачають активне використання текстових даних у різних галузях, починаючи від програмування та документообігу й закінчуючи освітою, наукою та засобами масової комунікації. Обробка текстових файлів є невід’ємною частиною повсякденної діяльності користувачів, редакторів, розробників програмного забезпечення та аналітиків. У зв’язку з цим виникає потреба у створенні простих, надійних й ефективних утиліт, здатних швидко виконувати аналіз тексту. Одним із найпоширеніших завдань є визначення кількості слів і символів, що необхідно для статистичного аналізу, контролю обсягу документів, підготовки звітів, форматування публікацій або створення текстів із заданими параметрами.

Актуальність обраної теми полягає в тому, що стандартні інструменти підрахунку слів та символів не завжди доступні або зручні. Крім того, багато існуючих рішень працюють лише в певних середовищах або не підтримують обробку різних типів файлів. У межах дисципліни «Системне програмування» важливо вміти створювати утиліти, які взаємодіють із файловою системою, здійснюють обробку даних на низькому рівні та можуть бути інтегровані в інші програмні засоби. Тому розробка утиліти для підрахунку кількості слів і символів у текстовому файлі є доцільним та практично значущим завданням.

Згідно з вимогами завдання на курсовий проєкт, необхідно створити програмний продукт, здатний зчитувати текстовий файл, аналізувати його вміст, коректно визначати кількість слів, символів, пробілів та, за потреби, інших елементів. На основі аналізу вимог формуються додаткові технічні умови: підтримка стандартних форматів текстових файлів, стабільність роботи з великими обсягами даних, коректне трактування розділових знаків, мінімальні ресурси системи та зручність використання. Програма має забезпечувати простий інтерфейс взаємодії – консольний або графічний – і працювати у середовищі операційної системи *Windows*.

Метою дослідження є розробка утиліти, яка забезпечує автоматизований підрахунок кількості слів і символів у текстовому файлі з урахуванням вимог

системного програмування. Для досягнення мети необхідно вирішити низку завдань:

- проаналізувати формат текстових файлів та способи їх обробки;
- визначити системні вимоги до програмної утиліти;
- визначити критерії підрахунку слів і символів;
- обрати мову програмування та середовище розробки;
- реалізувати алгоритм обробки вмісту файлу;
- розробити інтерфейс взаємодії з користувачем;
- розробити керівництво користувача.

Методами дослідження виступають аналіз специфікацій і стандартів представлення тексту, планування та розробка програмної архітектури, застосування засобів системного програмування, модульне тестування та експериментальна перевірка роботи утиліти.

Практичне значення проєкту полягає у створенні програмного продукту, який може бути використаний у повсякденній роботі студентів, викладачів, редакторів і розробників програмного забезпечення. Утиліта здатна виконувати функції самостійного інструмента чи частини більшого програмного комплексу. Її можна інтегрувати в середовища для обробки текстів, використовувати для перевірки обсягу документів або автоматизації редагування.

Таким чином, у межах даного курсового проєкту передбачено реалізацію утиліти, що поєднує функціональність, простоту використання та відповідність вимогам дисципліни «Системне програмування». Отриманий результат матиме як навчальну, так і практичну цінність, оскільки дозволяє закріпити знання й навички роботи з файлами, системними ресурсами та обробкою текстових даних.

РОЗДІЛ 1. ТЕОРЕТИЧНЕ ПРОЕКТУВАННЯ

1.1 Аналіз існуючих аналогів програмного продукту

У сучасних умовах зростання обсягів цифрової інформації з'являється потреба у швидкому аналізі текстових даних для подальшої обробки, редагування або статистичного оцінювання. Зокрема, підрахунок кількості слів та символів є однією з базових операцій у роботі копірайтерів, редакторів, програмістів, науковців та студентів. Багато текстових редакторів містять подібний функціонал, однак не завжди забезпечують зручність використання, достатню точність або можливість автоматизації обробки файлів. Саме тому створення спеціалізованої утиліти для підрахунку текстових параметрів є актуальним і практично значущим завданням.

Перед початком безпосередньої розробки програмного продукту необхідно дослідити вже наявні рішення на ринку програмного забезпечення. Це дозволить виявити їх переваги та обмеження, визначити оптимальні підходи до реалізації, а також сформулювати перелік функціональних вимог до майбутньої утиліти.

На Рисунку 1.1 представлено головну сторінку онлайн-сервісу *Charcounter*, який призначений для підрахунку кількості символів і слів у введеному тексті. У центрі сторінки розміщено велике текстове поле, куди користувач може вставити або набрати будь-який текст. Нижче автоматично відображаються результати аналізу: загальна кількість символів, кількість слів, символів без пробілів та кількість абзаців. Таким чином, сервіс виконує базову функцію оперативного аналізу тексту без потреби встановлення додаткового програмного забезпечення.

Попри простоту та швидкодію, даний сервіс має низку недоліків. По-перше, він працює тільки з текстом, введеним вручну, і не підтримує завантаження файлів, що обмежує його використання при обробці великих документів. По-друге, інтерфейс хоч і мінімалістичний, але не передбачає розширених налаштувань, таких як виключення чисел, спеціальних символів, аналіз частоти слів чи збереження результатів. Також відсутня кнопка швидкого очищення поля, на що вказують навіть користувачі у розділі зворотного зв'язку. Крім того, сервіс

повністю залежить від підключення до Інтернету, що робить його незручним для автономного використання.

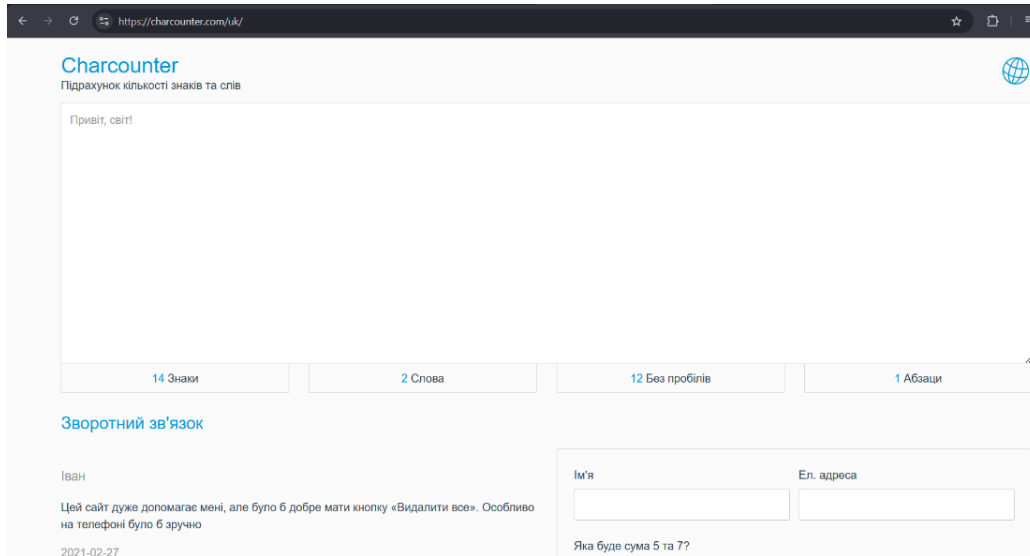


Рисунок 1.1 – Головна сторінка *Charcounter*

На Рисунку 1.2 представлено онлайн-сервіс *GroupDocs Words Counter*, призначений для підрахунку слів у *PDF-документах*. Центральний елемент інтерфейсу - це велика область завантаження файлів, куди користувач може натиснути або перетягнути документ для аналізу. У верхній частині сторінки розміщено логотип платформи та меню з основними розділами, що додає сайту впорядкованості та зручності у навігації. У нижній частині є додаткові кнопки для роботи з іншими форматами, такими як *Word* або введення вручну текст.

Основною функцією цього веб-додатка є автоматичний підрахунок кількості слів і символів у *PDF-файлах*. Сервіс працює повністю онлайн, що дозволяє

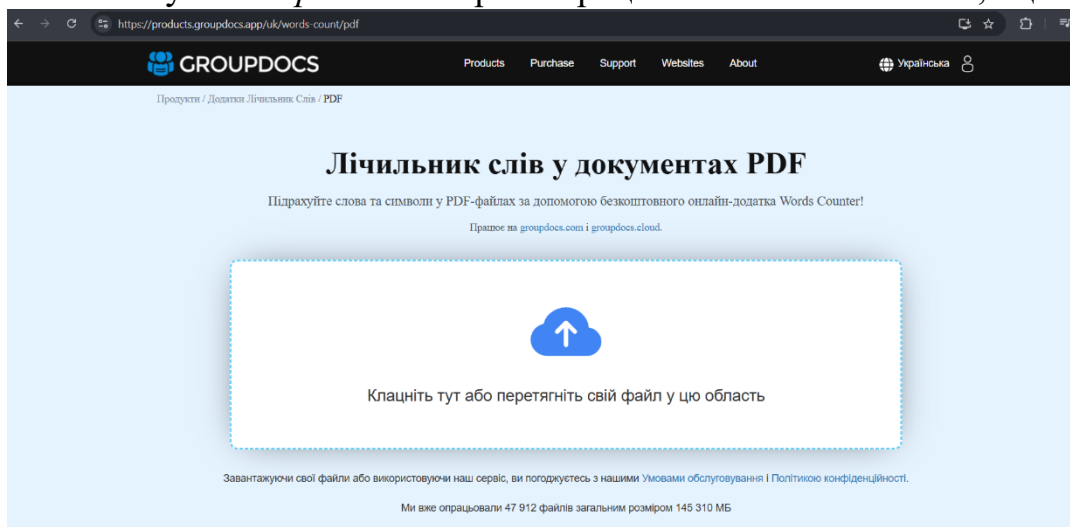


Рисунок 1.2 – Головна сторінка *GroupDocs Words Counter*

швидко виконати аналіз без встановлення додаткових програм. Також він підтримує завантаження файлів у різний спосіб – через натискання або *drag-and-drop*, що робить його зручним навіть для недосвідчених користувачів.

До основних переваг можна віднести простий інтерфейс, швидкість роботи та підтримку різних форматів документів. Сервіс не потребує реєстрації чи встановлення програмного забезпечення, що значно спрощує його використання.

Водночас платформа має і недоліки. Для її роботи обов'язково потрібне інтернет-з'єднання, а завантаження особистих документів на сторонній сервер може бути ризикованим з точки зору конфіденційності. Крім того, функціонал обмежується лише підрахунком слів і символів – сервіс не надає розширеної статистики або можливості редагування текстів.

1.2 Мова програмування та середовище розробки

У процесі розробки програмного забезпечення важливим етапом є вибір відповідної мови програмування та середовища розробки, оскільки саме вони визначають зручність реалізації функціоналу, продуктивність системи та можливості її подальшого розширення. Для створення утиліти підрахунку слів і символів було обрано C++.

C++ – це потужна мова програмування високого рівня, яка підтримує кілька парадигм, зокрема об'єктно-орієнтоване, узагальнене (шаблонне) та процедурне програмування. Вона була створена як розвиток мови C, зберігши її високу продуктивність та гнучкість, але водночас доповнивши більш зручними засобами для побудови складних програмних систем. Основною інновацією C++ є підтримка класів, які дозволяють поєднувати дані та методи їх обробки в єдині структури, реалізуючи принципи інкапсуляції, успадкування та поліморфізму.

Мова відзначається суворою типізацією, що забезпечує більшу надійність коду, оскільки кожна змінна повинна мати явно визначений тип. Однією з важливих особливостей C++ є можливість низькорівневого доступу до пам'яті через вказівники та механізми динамічного виділення пам'яті. Завдяки цьому C++ широко застосовується для розробки операційних систем, ігрових рушіїв,

прикладних програм з високими вимогами до швидкодії та вбудованих систем. Мова стандартизується міжнародним комітетом, що забезпечує її стабільний розвиток.

Синтаксис *C++* побудований на основі синтаксису *C* та є чутливим до регістру. Вихідний код складається з лексем – таких як ключові слова, ідентифікатори, оператори та літерали. Пробіли, символи табуляції та переходи на новий рядок використовуються як розділювачі між лексемами і, зазвичай, ігноруються компілятором. Кожна інструкція в мові повинна завершуватися крапкою з комою, а групи інструкцій, наприклад у функціях чи керуючих конструкціях, об'єднуються у блоки за допомогою фігурних дужок.

C++ підтримує два типи коментарів – однорядкові та багаторядкові, що дозволяє документувати код. Перед початком компіляції обробляються директиви препроцесора, які починаються із символу *#*, наприклад *#include* використовується для підключення заголовних файлів. Функції в *C++* завжди мають вказувати тип значення, яке вони повертають. Для уникнення конфліктів імен використовуються простори імен (*namespace*), а для організації об'єктно-орієнтованої структури програм визначаються класи з розмежуванням доступу до їхніх членів за допомогою специфікаторів *public*, *private* та *protected*.

Як основне середовище розробки використано *Visual Studio 2022*, що забезпечує зручні інструменти для написання, налагодження та тестування коду.

Visual Studio 2022 – це потужне інтегроване середовище розробки (*IDE*), створене компанією *Microsoft* для професійного створення програмного забезпечення. Воно надає розробникам повний набір інструментів для написання, компіляції, тестування та налагодження програм, підтримуючи різні мови програмування, зокрема *C++* та *.NET*.

Однією з найважливіших змін у цій версії є перехід на 64-розрядну архітектуру, що дозволяє *IDE* використовувати значно більше оперативної пам'яті. Завдяки цьому *Visual Studio 2022* працює швидше та стабільніше навіть із дуже великими проєктами, усуваючи типові помилки переповнення пам'яті, які виникали в попередніх версіях.

Серед ключових можливостей варто виділити розширену підтримку C++, зокрема покращену сумісність зі стандартом C++20, удосконалене автодоповнення коду (*IntelliSense*) та інструменти оптимізації продуктивності. Функція *Hot Reload* дозволяє вносити зміни в код під час виконання програми без необхідності її повторного запуску, що значно прискорює процес розробки.

Visual Studio 2022 також використовує технології штучного інтелекту (*IntelliCode*) для контекстних підказок, здатних автоматично пропонувати не лише окремі слова, а й цілі конструкції коду. Крім того, у середовище вбудовані інструменти для роботи з *Git*, які спрощують керування репозиторіями, гілками та спільну розробку.

Не менш важливими є розширені можливості налагодження та діагностики, що дозволяють аналізувати роботу додатків у реальному часі та виявляти проблеми продуктивності. Крім того, *Visual Studio 2022* підтримує кросплатформенну розробку, надаючи засоби для створення мобільних і десктопних застосунків на *.NET MAUI*, а також компіляцію й тестування C++-проектів для *Linux*.

1.3 Постановка завдання

Метою курсового проєкту є розробка програмної утиліти для підрахунку кількості слів і символів у текстовому файлі, яка дозволить користувачу швидко отримувати статистичну інформацію про вміст текстових документів. Такий функціонал є актуальним для письменників, журналістів, студентів, викладачів, редакторів, копірайтерів та інших фахівців, що працюють з великими обсягами тексту й потребують контролю обсягу написаного матеріалу.

Програма повинна бути реалізована мовою C++ із використанням інтегрованого середовища розробки *Visual Studio 2022*. Рішення має працювати з файлами у форматі *.txt*, відкривати їх у режимі читання, виконувати аналіз вмісту та відображати результати у зручному для користувача вигляді.

Основні вимоги до функціоналу програми:

1) Здійснення підрахунку:

– загальної кількості символів;

- кількості символів без пробілів;
 - кількості слів;
- 2) Обробка як коротких, так і великих файлів (обсягом від кількох байтів до кількох мегабайтів).
 - 3) Коректне розпізнавання слів, включаючи випадки з декількома пробілами, розділовими знаками та табуляцією.
 - 4) Виведення результатів у зрозумілому форматі.

Додатково бажано передбачити:

- 1) валідацію шляху до файлу та обробку помилок (наприклад, файл не знайдено або не може бути відкритий);
- 2) можливість повторної обробки іншого файлу без перезапуску програми;

Структура програми повинна відповідати принципам модульності та читабельності коду. Для зберігання та відображення результатів варто використовувати окремі функції або класи. Ретельно прокоментований код є обов'язковою вимогою.

Очікуваним результатом курсового проєкту є готова утиліта, здатна швидко й точно виконувати статистичний аналіз текстових файлів, а також пояснювальна записка з описом алгоритму роботи, вибору технологій та структури програми.

РОЗДІЛ 2. РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 UML-діаграми для реалізації програми

У процесі розробки програмного забезпечення важливо не лише створити правильний код, а й ретельно продумати архітектуру програми, взаємодію її складових та логіку виконання. Для цього застосовуються *UML-діаграми (Unified Modeling Language)*, які надають можливість наочно відобразити структуру системи, взаємозв'язки між класами, об'єктами, функціями та взаємодію користувача з програмою.

Завдяки таким діаграмам можна ще до початку програмування краще зрозуміти, як працюватиме система, спростити процес її проєктування, полегшити відлагодження та забезпечити узгодженість між усіма етапами розробки.

Діаграма варіантів використання (або діаграма прецедентів, англ. *Use Case Diagram*) - це один із типів *UML-діаграм*, що відображає взаємозв'язки між користувачами системи (акторами) та її функціональними можливостями (прецедентами).

Вона демонструє різні сценарії взаємодії з системою та типи користувачів, які з нею працюють. Прецеденти зазвичай позначаються еліпсами або колами, а актори - стилізованими фігурами у вигляді паличок. Такі діаграми часто використовуються разом з іншими *UML-діаграмами* для повнішого опису структури та поведінки системи.

На Рисунку 2.1 зображено діаграму варіантів використання (*Use Case Diagram*) для програми підрахунку слів і символів.

Діаграма представляє єдиного актора - Користувача (*User*). Блакитний прямокутник позначає межі системи, яка називається «Програма підрахунку слів і символів».

Основний потік роботи програми складається з послідовності варіантів використання:

1. Вказати шлях до файлу.
2. Відкрити файл.

3. Порахувати кількість символів.
4. Порахувати кількість слів.
5. Вивести результати на екран.
6. Закрити вікно програми.

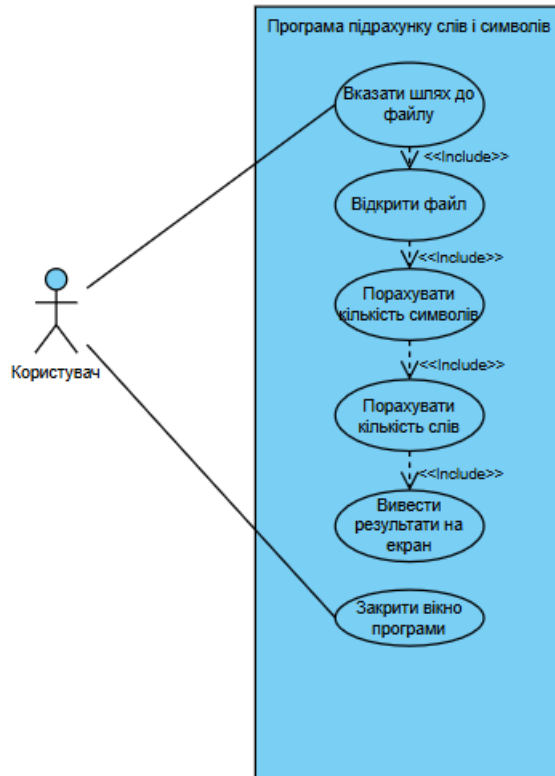


Рисунок 2.1 – Діаграма варіантів використання

Зв'язки між цими кроками позначені стереотипом <<include>>, що означає, що один варіант використання обов'язково включає функціональність іншого. Це вказує на послідовність обов'язкових дій, які становлять роботу програми. Користувач взаємодіє із системою для ініціювання цього процесу (зокрема, вказує шлях до файлу та ініціює запуск підрахунку та закриття програми).

Діаграма діяльності (*Activity Diagram*) – це розширений різновид блок-схеми, який використовується для візуалізації процесу використання системи. Вона ілюструє потік виконання від однієї дії до іншої, демонструючи цілісну роботу системи.

Крім того, варто додати, що діаграма діяльності є особливо корисною для моделювання бізнес-процесів і паралельного виконання (коли кілька дій відбуваються одночасно). Вона використовує спеціальні елементи (наприклад,

розгалуження (*fork*) та злиття (*join*)) для деталізації того, як процес може розділятися на паралельні потоки, а потім знову об'єднуватися. Це робить її

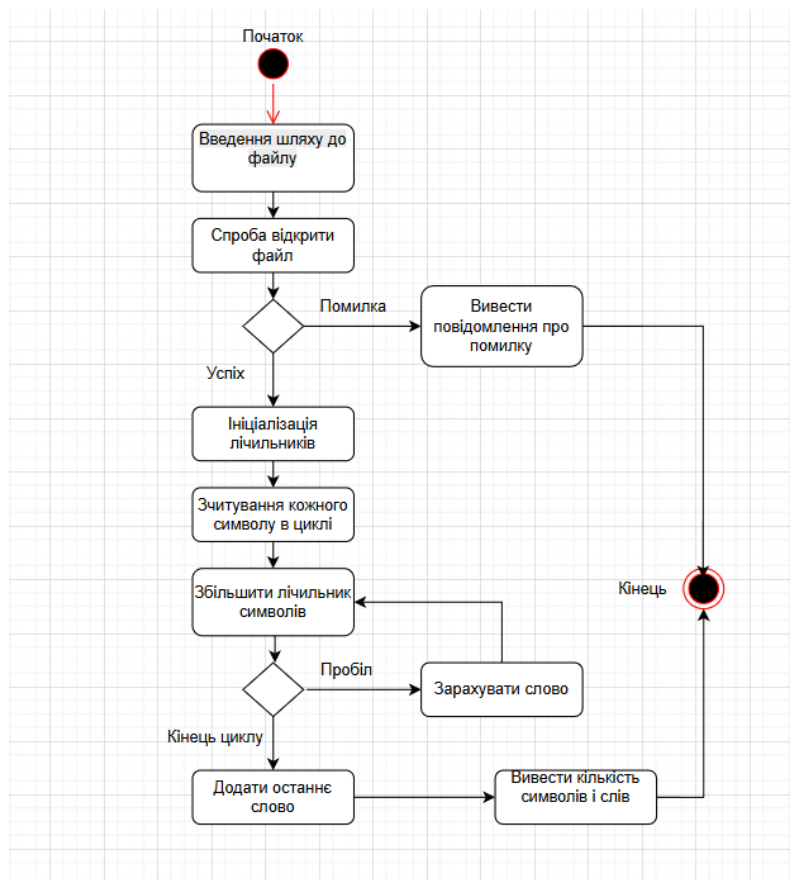


Рисунок 2.2 – Діаграма діяльності

потужним інструментом не лише для показу простої послідовності, але й для аналізу складної логіки та оптимізації робочих процесів.

На рисунку 2.2 зображено діаграму діяльності, яка детально візуалізує алгоритм роботи програми для підрахунку слів і символів. Вона починається з вузла «Початок» і описує послідовність кроків та рішень.

Процес стартує з дії «Введення шляху до файлу», за якою слідує «Спроба відкрити файл». Після цього відбувається керуюче рішення, яке перевіряє результат спроби. Якщо виникає «Помилка», процес переходить до дії «Вивести повідомлення про помилку» і відразу завершується вузлом «Кінець». Якщо ж спроба успішна (файл відкрито), процес продовжується до етапу підрахунку.

Успішне відкриття файлу веде до «Ініціалізації лічильників» (встановлення початкових значень для кількості слів і символів). Далі вступає в дію основний

цикл обробки: «Зчитування кожного символу в циклі». Після зчитування кожного символу обов'язково виконується дія «Збільшити лічильник символів».

Після збільшення лічильника символів відбувається друге керуюче рішення, яке перевіряє, чи є щойно зчитаний символ пробілом. Якщо це пробіл, виконується дія «Зарахувати слово» і процес повертається до збільшення лічильника символів. Якщо символ не пробіл, процес переходить до перевірки «Кінець циклу» (перевірка, чи досягнуто кінця файлу). Якщо це не кінець циклу, процес повертається до зчитування наступного символу. Після завершення циклу (коли досягнуто кінця файлу) виконується дія «Додати останнє слово» (оскільки останнє слово у файлі, як правило, не закінчується пробілом). Фінальним кроком перед завершенням є «Вивести кількість символів і слів», після чого програма досягає вузла «Кінець».

Діаграма класів (*Class diagram*) – це ключова статична діаграма в *UML*, що відображає структуру моделі системи в термінах об'єктно-орієнтованого програмування. Вона показує основні декларативні елементи: класи, інтерфейси, типи даних та їхній вміст (атрибути й операції), а також відносини (зв'язки) між ними. Хоча діаграма зосереджена на статичі, вона може включати позначення для організації елементів.

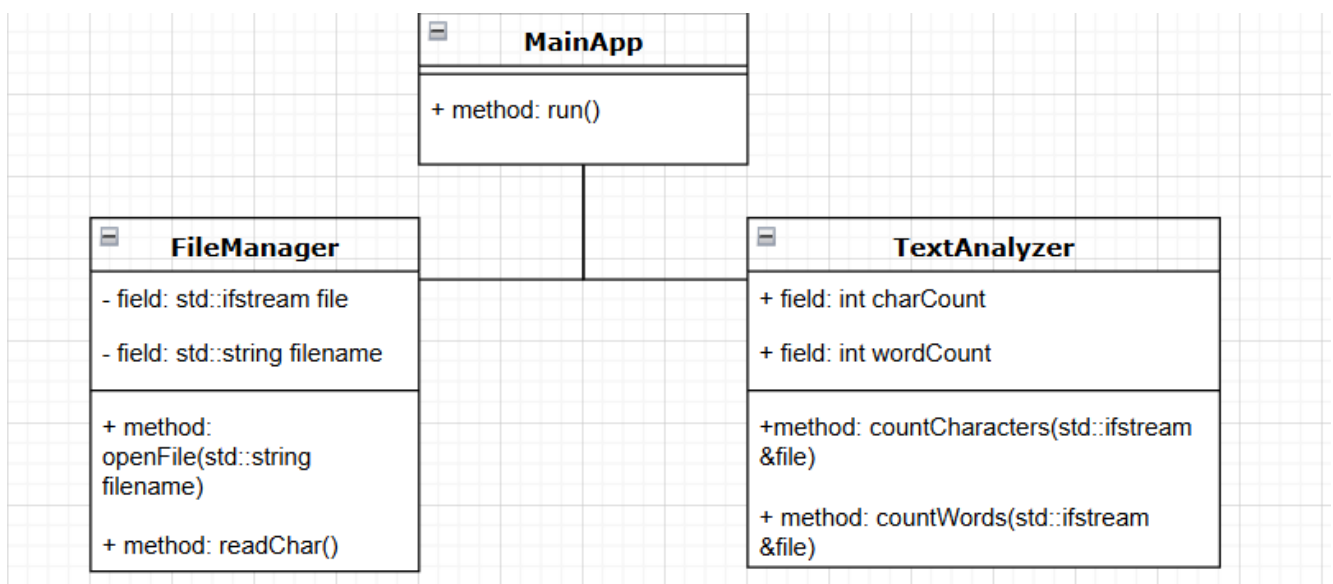


Рисунок 2.3 - Діаграма класів

На Рисунку 2.3 зображено діаграму класів для програми підрахунку слів і символів. Ця діаграма відображає статичну структуру системи, поділяючи її функціональність на три основні класи та показуючи зв'язки між ними.

MainApp (Головний додаток): Це головний клас, який слугує точкою входу в програму. Він містить єдиний публічний метод *run()*, який ініціює та координує виконання всіх операцій (введення шляху, відкриття файлу, аналіз та виведення результатів).

FileManager: Цей клас відповідає за роботу з файловою системою. Він має два приватні поля: *file: std::ifstream file* (потік для читання файлу) та *filename: std::string filename* (ім'я файлу). Його публічні методи включають *openFile(std::string &filename)* для відкриття вказаного файлу та *readChar()* для зчитування наступного символу з відкритого потоку, що відповідає за низькорівневу операцію вводу-виводу.

TextAnalyzer: Цей клас відповідає за логіку підрахунку. Він містить два публічні поля: *charCount: int* та *wordCount: int*, які зберігають результати підрахунку символів і слів. Його ключові методи – *countCharacters(std::ifstream &file)* та *countWords(std::ifstream &file)* – приймають потік до файлу як аргумент і виконують фактичні операції підрахунку, реалізуючи бізнес-логіку програми.

Між класами встановлені відносини асоціації, що вказують на залежність і взаємодію. Клас *MainApp* пов'язаний як із *FileManager*, так і з *TextAnalyzer*. Це означає, що *MainApp* використовує (або володіє екземплярами) обома цими класами для виконання своєї функції *run()*: спершу він використовує *FileManager* для доступу до даних, а потім передає доступ до цих даних *TextAnalyzer* для обробки. Це відображає чисту архітектуру, де відповідальність чітко розділена: один клас відповідає за доступ до даних, інший – за їх аналіз, а третій – за координацію.

2.2 Програмна реалізація методів

Бібліотеки

`#include <iostream>` - це директива препроцесора в C++, яка підключає заголовковий файл `iostream`, що містить функції та класи для введення-виведення даних, таких як читання з клавіатури або виведення на екран.

`#include <fstream>` - це директива препроцесора в C++, яка підключає заголовковий файл `fstream`, що надає засоби для роботи з файлами через потоки введення/виведення.

`#include <string>` - це директива препроцесора в C++, яка підключає стандартну бібліотеку рядків (`std::string`), надаючи можливість роботи з текстом у програмі.

`#include <cctype>` - це директива у програмах C++ містить заголовковий файл `<cctype>`, який надає набір функцій для класифікації та перетворення символів.

Ініціалізація та введення

Програма починається з функції `int main()`, яка є обов'язковою точкою входу. У ній створюється змінна `std::string filename`, призначена для зберігання шляху, який вводить користувач. За допомогою `std::cout` виводиться запит для введення шляху до текстового файлу, а функція `std::getline(std::cin, filename)` зчитує введений шлях повністю, включаючи можливі пробіли. Це гарантує, що шлях до файлу буде коректно сприйнятий, навіть якщо він містить складні каталоги.

Відкриття файлу та обробка помилок

Наступний критичний крок – відкриття файлового потоку для читання: `std::ifstream file(filename)`. Цей об'єкт намагається встановити зв'язок із зазначеним файлом. Одразу після цього відбувається перевірка `if (!file)`. Якщо потік `file` не знаходиться в робочому стані (наприклад, файл не знайдено, або доступ до нього заборонено), програма виводить повідомлення про помилку в потік `std::cerr` (призначений для виведення помилок) і негайно завершує роботу, повертаючи код 1, що сигналізує операційній системі про невдаче виконання.

Підготовка до підрахунку та головний цикл

Якщо файл успішно відкрито, ініціалізуються лічильники: `int charCount = 0` для символів та `int wordCount = 0` для слів, а також тимчасовий буфер `std::string`

`word` для накопичення символів поточного слова. Основна робота відбувається у циклі `while (file.get(ch))`. Цей цикл послідовно зчитує кожен символ із файлу у змінну `ch` доти, доки потік не досягне кінця або не виникне помилка читання. На початку кожної ітерації лічильник `charCount` безумовно збільшується, враховуючи всі символи файлу.

Логіка підрахунку слів

Усередині циклу реалізована логіка виявлення слів. Вона базується на перевірці, чи є поточний символ `ch` пробільним (за допомогою `std::isspace(ch)`).

- 1) Якщо символ є пробільним, програма перевіряє, чи не порожній буфер `word`. Якщо `word` містить символи, це означає, що слово щойно закінчилося, тому `wordCount` збільшується, а буфер `word` очищується (`word.clear()`) для прийому наступного слова.
- 2) Якщо символ не пробільний, він додається до тимчасового буфера `word` (`word += ch`), продовжуючи накопичення поточного слова.

Завершення та виведення результатів

Після того, як цикл зчитування завершується (досягнуто кінця файлу), програма виконує фінальну перевірку: `if (!word.empty()) { wordCount++; }`. Цей блок необхідний для обліку останнього слова у файлі, якщо воно не завершується пробільним символом (наприклад, переведенням рядка). Далі файловий потік `file.close()` закривається. Нарешті, результати підрахунку символів і слів виводяться в консоль, і програма успішно завершується, повертаючи код 0.

Повний вихідний код програми Див Додаток А.

РОЗДІЛ 3. КЕРІВНИЦТВО КОРИСТУВАЧА

У цьому розділі представлено покроковий посібник з використання програми на C++ для автоматичного підрахунку слів і символів у текстових файлах. Описано всі етапи роботи з утилітою: встановлення, запуск, використання та налагодження. Метою є надання користувачам повного розуміння того, як ефективно аналізувати текстові дані та отримувати точні результати підрахунків.

Далі наведена детальна покрокова інструкція, як завантажити, створити та запустити цю програму у *Visual Studio 2022* під *Windows*.

1. Встановлення *Visual Studio 2022*.

Для підготовки робочого середовища, якщо у вас ще не встановлено *Visual Studio 2022*, необхідно виконати такі кроки: перейдіть на офіційний сайт та завантажте безкоштовну версію *Visual Studio Community 2022*. Під час інсталяції критично важливо вибрати необхідне робоче навантаження (*workload*): «*Desktop development with C++*» (розробка класичних програм для *Windows* з використанням C++). Вибір цього навантаження гарантує, що у вашій системі будуть встановлені всі потрібні бібліотеки, заголовні файли та компілятор *MSVC*, необхідні для успішної компіляції програми.

2. Створення нового проєкту в *Visual Studio*.

Для початку роботи відкрийте *Visual Studio 2022* та оберіть опцію «*Create a new project*» (Створити новий проєкт). У вікні вибору шаблону введіть у полі пошуку «*Console App*» і виберіть шаблон «*Console App*»(C++), після чого натисніть *Next*. На наступному етапі задайте ім'я проєкту, наприклад, «*Utility*», у полі *Project name*. Виберіть зручну для вас теку збереження і завершіть процес, натиснувши *Create*.

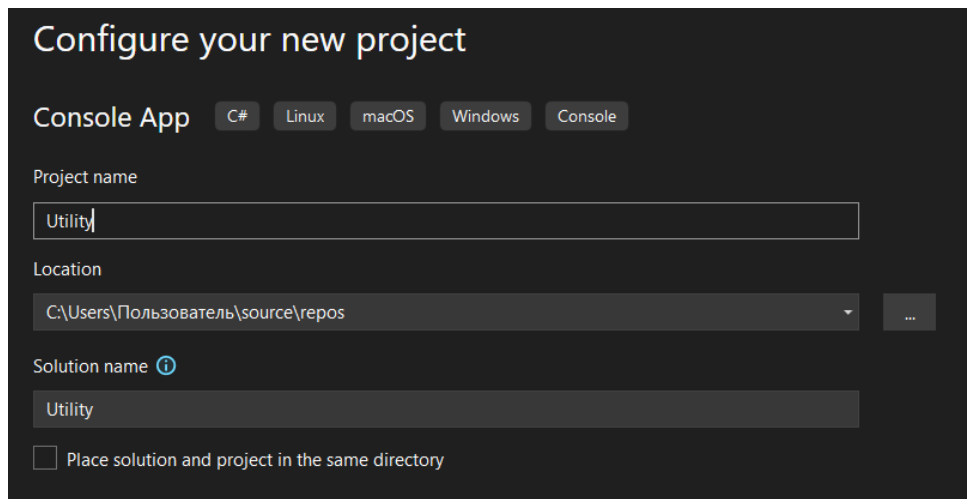


Рисунок 3.1 - Приклад створення нового проєкту в VS 2022

3. Додавання вихідного коду програми.

Після успішного створення проєкту, *Visual Studio* автоматично згенерує основний файл, названий, *Utility.cpp*. Знайдіть та відкрийте цей файл (він відображатиметься у вікні *Solution Explorer* ліворуч). Видаліть увесь початковий код, який був автоматично вставлений *Visual Studio*. Замість нього скопіюйте та вставте в цей файл фінальний код програми, який наведений в Додатку А та додатково наданий в форматі *.cpp*. Завершіть цей етап, натиснувши *Ctrl + S*, щоб зберегти всі внесені зміни у файлі.

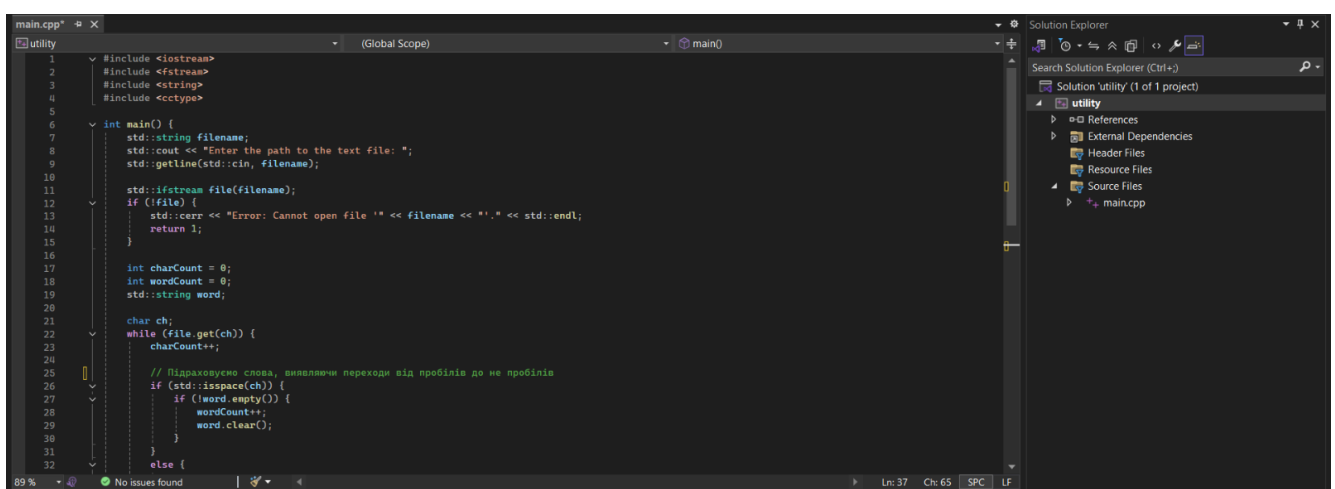


Рисунок 3.2 – Вигляд VS 2022 після додавання коду

4. Підготовка вхідного файлу для аналізу.

Створіть будь-який текстовий файл, який буде використовуватися для тестування програми. Як приклад, можна створити файл за шляхом: *D:\checking.txt*.

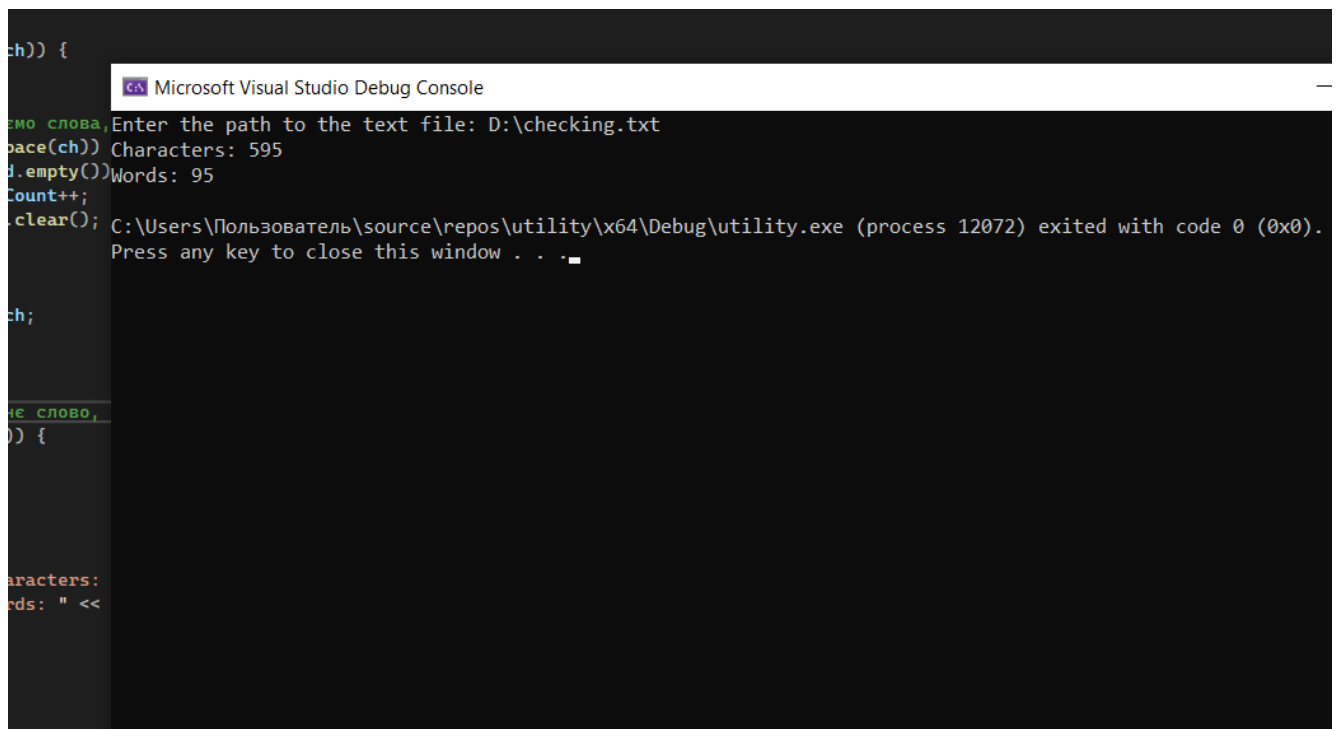
Наповніть його кількома рядками довільного тексту, наприклад *Lorem Ipsum* - це текст-«риба».

Збережіть цей файл і обов'язково запам'ятайте повний шлях до нього, оскільки він знадобиться для введення в програму під час її запуску.

5. Запуск програми та тестування

Щоб запустити програму, скористайтесь зеленим трикутником «*Local Windows Debugger*» у *Visual Studio* або натисніть клавішну комбінацію *Ctrl + F5* (для запуску без режиму налагодження). Це відкриє вікно консолі, де ви побачите запит: *Enter the path to the text file:*

У цьому полі введіть повний шлях до підготовленого вами текстового файлу (наприклад, *D:\checking.txt*) і натисніть *Enter*. Після обробки файлу програма виведе фінальний результат підрахунку символів та слів.

The image shows a screenshot of the Microsoft Visual Studio Debug Console. The console window has a title bar that says "Microsoft Visual Studio Debug Console". The text inside the console is as follows:

```
Enter the path to the text file: D:\checking.txt
Characters: 595
Words: 95
C:\Users\Пользователь\source\repos\utility\x64\Debug\utility.exe (process 12072) exited with code 0 (0x0).
Press any key to close this window . . .
```

On the left side of the image, a portion of the C++ source code is visible, showing a function that processes text and counts characters and words.

Рисунок 3.3 – Результат роботи програми

6. Приклад помилки в роботі програми

В разі неправильно введеного шляху до файлу, програма повідомить про помилку та не зможе відкрити файл. Після цього необхідно закрити консоль відладки, ще раз запустити компіляцію програми за допомогою комбінації клавіш



Рисунок 3.4 – Приклад помилки роботи програми

Ctrl + *F5* та повторно ввести правильний шлях до файлу, підрахунок слів та символів в якому необхідно провести.

ВИСНОВКИ

У ході виконання курсового проєкту на тему «Розробка утиліти для підрахунку кількості слів і символів у текстовому файлі» було розроблено прикладне програмне забезпечення, призначене для автоматизованого аналізу текстових документів. Основною метою роботи було створення простої та надійної утиліти, яка дає змогу користувачу швидко отримувати статистичну інформацію про обсяг тексту – зокрема кількість слів і символів, що містяться у файлі.

Актуальність проєкту обґрунтовується тим, що, на відміну від багатофункціональних текстових редакторів (як-от *Microsoft Word*), які мають складніший інтерфейс і орієнтовані на ширший спектр завдань, розроблена програма зосереджена виключно на конкретній задачі, що робить її зручнішою та легшою у використанні.

Програму було реалізовано мовою C++, що забезпечило високу швидкість виконання та ефективне використання ресурсів. Для роботи з даними використовувалися лише стандартні бібліотеки (*<iostream>*, *<fstream>*, *<string>*), що гарантує кросплатформність рішення та відсутність зовнішніх залежностей. Ключовим елементом реалізації є посимвольний алгоритм зчитування, який точно підраховує символи та ідентифікує слова на основі переходів між пробільними та непробільними символами. Такий підхід забезпечив коректний облік слів незалежно від форматування, включно з обробкою випадків, коли файл не закінчується пробілом.

Під час виконання проєкту були відпрацьовані практичні навички програмування на C++, роботи з файлами, використання потоків вводу-виводу, розробки алгоритмів послідовної обробки текстових даних, а також навички налагодження програм у середовищі *Visual Studio 2022*. У ході роботи було реалізовано всі поставлені завдання, зокрема:

- створено алгоритм підрахунку кількості символів і слів;
- реалізовано перевірку правильності відкриття файлу;

- забезпечено коректне завершення програми при виникненні помилок;
- протестовано роботу програми з різними текстовими файлами.

Результати виконання курсового проєкту підтвердили, що поставлену мету досягнуто повністю. Розроблена утиліта є простою у використанні, не потребує складного встановлення та може бути застосована як окремий програмний інструмент або як допоміжний модуль у більших системах аналізу тексту.

Отже, під час розробки було створено ефективне програмне забезпечення, яке вирішує практичну задачу підрахунку текстових характеристик і демонструє приклад застосування основних принципів обробки даних у мові C++. Програма має потенціал для подальшого вдосконалення — зокрема, можна реалізувати графічний інтерфейс користувача, додати підтримку різних кодувань, статистику частотності слів або функцію порівняння текстів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ:

1. <https://products.groupdocs.app/uk/words-count/pdf>
2. <https://charcounter.com/uk/>
3. http://www.znannya.org/?view=C++_basics
4. <https://uk.wikipedia.org/wiki/C%2B%2B>
5. <https://learn.microsoft.com/uk-ua/visualstudio/ide/?view=vs-2022>
6. <https://learn.microsoft.com/uk-ua/visualstudio/get-started/visual-studio-ide?view=vs-2022>
7. <https://online.visual-paradigm.com/app/diagrams/#diagram:proj=0&type=UseCaseDiagram&width=11&height=8.5&unit=inch>
8. https://uk.wikipedia.org/wiki/%D0%94%D1%96%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%B0_%D0%BF%D1%80%D0%B5%D1%86%D0%B5%D0%B4%D0%B5%D0%BD%D1%82%D1%96%D0%B2
9. <https://www.drawio.com/>
10. <https://dou.ua/forums/topic/40575/>
11. <https://uk.wikipedia.org/wiki/Iostream#:~:text=iostream%20%E2%80%94%D0%B1%D1%96%D0%B1%D0%BB%D1%96%D0%BE%D1%82%D0%B5%D0%BA%D0%B0%20%D1%96%20%D0%B2%D1%96%D0%B4%D0%BF%D0%BE%D0%B2%D1%96%D0%B4%D0%BD%D0%B8%D0%B9%20%D0%B7%D0%B0%D0%B3%D0%BE%D0%BB%D0%BE%D0%B2%D0%BD%D0%B8%D0%B9,%D0%B2%20%D0%BC%D0%BE%D0%B2%D1%96%20%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D1%83%D0%B2%D0%B0%D0%BD%D0%BD%D1%8F%20C++>
12. [https://itproger.com/ua/spravka/cpp/fstream#:~:text=C++%20fstream-%D0%A7%D1%82%D0%BE%20%D1%82%D0%B0%D0%BA%D0%BE%D0%B5%20fstream%20%D0%B2%20C++?,\(%D0%B4%D0%BB%D1%8F%20%D1%87%D1%82%D0%B5%D0%BD%D0%B8%D1%8F%20%D0%B8%20%D0%B7%D0%B0%D0%BF%D0%B8%D1%81%D0%B8\)](https://itproger.com/ua/spravka/cpp/fstream#:~:text=C++%20fstream-%D0%A7%D1%82%D0%BE%20%D1%82%D0%B0%D0%BA%D0%BE%D0%B5%20fstream%20%D0%B2%20C++?,(%D0%B4%D0%BB%D1%8F%20%D1%87%D1%82%D0%B5%D0%BD%D0%B8%D1%8F%20%D0%B8%20%D0%B7%D0%B0%D0%BF%D0%B8%D1%81%D0%B8))
13. <https://apps.microsoft.com/detail/xpdcfjdklzl8?hl=uk-UA&gl=ME>
14. <https://uk.lipsum.com/>

ДОДАТОК А

```
#include <iostream>
#include <fstream>
#include <string>
}
```